UNIVERSITÉ LIBANAISE

Faculté des Sciences Économiques et de Gestion Section III

Département: Informatique de Gestion
Matière: User Interface

Année Universitaire: 2024-2025
Niveau: M2 **Semestre:** S2
Responsable: Dr. Adolf Abdallah

Suggested Exam Solutions (Part 2)

Question 8: Differentiate between Usability, User Experience (UX), and User-Centered Design (UCD)

- **Usability:** Measures how easy and efficient an interface is for users to successfully accomplish specific tasks.
- **User Experience (UX):** Encompasses the overall emotional feeling, attitude, and satisfaction a user experiences before, during, and after interacting with a product.
- **User-Centered Design (UCD):** An iterative design process and philosophy that actively involves users at every development stage to shape the product around their precise needs.

Contribution to Building Better Interfaces:

UCD establishes the structured **process** to follow, Usability guarantees that the interface **functions effectively**, and UX ensures that the workflow is **valuable and satisfying** for the end user.

Question 9: UI Critique Using Norman's Seven Principles

Target System: University Student Portal

Critiques:

- **Poor Feedback:** When submitting a course registration form, the portal lacks a loading spinner or instant confirmation banner. The user remains completely uncertain if the system processed the action.
- **Weak Affordance:** The "Log Out" option at the bottom of the page renders as standard, flat text. It lacks any background bounding box, border, or underline to indicate it is an interactive, clickable link.

Suggested Improvements:

1. **Improve Feedback:** Embed an instantaneous confirmation toast or text notification (e.g., "Registration Successful!") alongside a clear loading animation directly upon form submission.
2. **Enhance Affordance:** Restyle the text element into a clear, distinct button outline with appropriate background padding so it visually commands interaction.

Question 10: Hierarchical Task Analysis (HTA) vs. ConcurTaskTree (CTT)

Key Differences:

- **Structure & Logic:** HTA decomposes user goals into a text-driven linear breakdown bound by descriptive "plans." CTT utilizes a formal, hierarchical graphical tree utilizing explicit temporal operators (such as choice, concurrency, and enabling).
- **Task Typology:** HTA handles all tasks universally without structural distinctions. CTT cleanly categorizes nodes into four explicit task allocations: User, System, Interaction, and Abstract.
- **Concurrency:** HTA cannot easily model overlapping tasks. CTT inherently supports complex concurrent actions.

When to Use Each:

- **Use HTA when:** Analyzing straightforward, rigid, sequential user workflows where actions follow a clear linear order (e.g., standard ATM cash withdrawals).
- **Use CTT when:** Designing highly interactive, multi-threaded modern computing applications where synchronous execution and human-machine cooperation are tightly coupled.

Question 11: Direct Manipulation Interfaces (DMI)

Advantages:

- **Easy to Learn and Retain:** Relies directly on physical, real-world analogies (e.g., dragging files), allowing rapid cognitive mastery and high retention.
- **Immediate Feedback:** Users witness instantaneous execution of actions, enabling immediate context tracking and fast error correction.
- **Reduced Anxiety:** Operations are easily reversible (via Undo commands), fostering safe system exploration.

Inappropriate Use Case Example:

Bulk or Programmatic Operations: Batch-renaming 1,000 document files or parsing large databases sequentially using mouse dragging is highly repetitive and inefficient. A programmatic script or Command Line Interface (CLI) is vastly superior here.

Question 12: Dialog Design Models Comparison

Model	Core Components	Ideal Use Case
State Transition Networks (STN)	Nodes (States) and directed Arrows (Events/ Transitions).	Simple, single-window linear screen sequences (e.g., basic setup wizards).
StateCharts	Hierarchy, Orthogonal Concurrency, History mechanisms, States, and Transitions.	Complex, reactive interfaces with simultaneous distinct windows (e.g., graphical editors).
Petri Nets	Places (States), Transitions (Events), Arcs, and dynamic Tokens.	Highly parallel workflows managing shared resources (e.g., collaborative multi-user applications).

Contrast Summary:

STNs encounter rapid readability degradation ("state explosion") under complex branching conditions. StateCharts counteract this by supporting internal hierarchies. While both StateCharts and Petri Nets natively capture concurrent operations, Petri Nets utilize token distribution to mathematically verify state safety and avoid race conditions.